

Backpropagation – Lab Report

Le Nghiem Thanh Thuy

1 Task of the source

The source reuse the neural network from lab 4, and add backpropagation with gradient descent so the network can learn weights and bias from data (not set by hand anymore).

The program have to do:

- Read the network shape from `structure.txt`, same as before.
- Initialize weights and bias randomly (in range $[-1, 1]$ so the hidden neurons not all act the same).
- Read a training set from a **CSV file**. Each row is: input values, then target output values.
- Train the network with backpropagation and stochastic gradient descent.
- Save the final weights and bias to a text file, same format as `parameters.txt`.

The activation is still sigmoid. The loss function is mean squared error

$$L = \frac{1}{2} \sum_k (o_k - t_k)^2.$$

2 Source design

2.1 What is new compare to lab 4

The class organization is the same: **NeuralNetwork** owns **Layer**, **Layer** owns **Neuron**. The new things are:

- Each **Neuron** now remember the last inputs and last output during forward, because backward need them.
- Each **Neuron** have `backward(d_output)` and `apply_gradients(lr)`.
- Each **Layer** have `backward(d_outputs)` – it call backward on every neuron and sum the gradient to send to the previous layer.
- **NeuralNetwork** have `train(csv_path, lr, epochs)` that read the CSV, do forward + backward + update for every sample, and print the loss every 1000 epochs.
- **NeuralNetwork** also have `save_parameters_to_file` so the trained weights can be reused later.

2.2 Backpropagation – the math

For one neuron with output $a = \sigma(z)$ where $z = \sum_i w_i x_i + b$:

- Sigmoid derivative: $\sigma'(z) = a(1 - a)$.
- If the upstream gives $\frac{\partial L}{\partial a}$, then

$$\delta = \frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot a(1 - a).$$

- Gradient for weights and bias:

$$\frac{\partial L}{\partial w_i} = \delta \cdot x_i, \quad \frac{\partial L}{\partial b} = \delta.$$

- Gradient to send to the previous layer (one term per input):

$$\frac{\partial L}{\partial x_i} = \delta \cdot w_i.$$

For the output layer with MSE loss, the starting gradient is simple:

$$\frac{\partial L}{\partial o_k} = o_k - t_k.$$

The Layer just sum the input-gradients from all its neurons, and pass that to the layer before. We repeat until we reach the input layer (the input layer have no weights, so we stop there).

2.3 Gradient descent update

After each sample, we update every parameter:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b},$$

with learning rate η . This is the stochastic (per-sample) version of gradient descent.

2.4 Flow

1. `Main.py` create a `NeuralNetwork` and call `build_from_file("structure.txt")`.
2. Call `initialize_random_weights()` so we start from random parameters.
3. Call `train("xor.csv", learning_rate=0.5, epochs=10000)`:
 - For each epoch, shuffle the samples.
 - For each sample: do `forward`, compute loss, do `backward`, then `apply_gradients`.
 - Print the loss every 1000 epochs to see if the network is learning.
4. Call `save_parameters_to_file("trained_parameters.txt")`.
5. Print the predicted output for every input in the training set, to check the result.

2.5 CSV input format

The CSV file have one sample per row. The first n columns are the inputs, the last m columns are the targets (n and m are decided by the shape in `structure.txt`). A header row is ok, it is skipped automatically.

Example for XOR (`xor.csv`):

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Table 1: XOR truth table used as the training set.

3 Result

3.1 XOR

The network shape is 2-2-1, learning rate 0.5, 10000 epochs, random seed 42. The loss start at about 2,914602 and go down to about 0.010692. All four inputs is classified correctly after training.

```
E:\Khoa hoc\USTH\DeepLearning\Week 2\Source\dl2026\5.Back_Propagation>python Main.py xor.csv
Building neural network from structure.txt
Initializing random weights...
Neural Network Structure:
Layer 1: 2 neurons
  Neuron 1: (input placeholder)
  Neuron 2: (input placeholder)
Layer 2: 2 neurons
  Neuron 1: Weights: [0.2789, -0.9500], Bias: -0.4499
  Neuron 2: Weights: [-0.5536, 0.4729], Bias: 0.3534
Layer 3: 1 neurons
  Neuron 1: Weights: [0.7844, -0.8261], Bias: -0.1562

Training on xor.csv with backpropagation + gradient descent...
Training with loss = BCE, lr = 0.1, epochs = 10000
Epoch   1 / 10000 - loss: 2.914602
Epoch 1000 / 10000 - loss: 1.878067
Epoch 2000 / 10000 - loss: 0.113066
Epoch 3000 / 10000 - loss: 0.052502
Epoch 4000 / 10000 - loss: 0.033914
Epoch 5000 / 10000 - loss: 0.024975
Epoch 6000 / 10000 - loss: 0.019738
Epoch 7000 / 10000 - loss: 0.016305
Epoch 8000 / 10000 - loss: 0.013882
Epoch 9000 / 10000 - loss: 0.012082
Epoch 10000 / 10000 - loss: 0.010692

Evaluation on training set:
input=[0.0, 1.0] target=[1.0] predicted=['0.9976'] -> [1]
input=[1.0, 1.0] target=[0.0] predicted=['0.0028'] -> [0]
input=[0.0, 0.0] target=[0.0] predicted=['0.0031'] -> [0]
input=[1.0, 0.0] target=[1.0] predicted=['0.9976'] -> [1]
```

Figure 1: Training log for XOR – loss decrease over epochs and final predictions.

```

Final parameters:
Neural Network Structure:
Layer 1: 2 neurons
  Neuron 1: (input placeholder)
  Neuron 2: (input placeholder)
Layer 2: 2 neurons
  Neuron 1: Weights: [7.3791, -7.5334], Bias: -4.0285
  Neuron 2: Weights: [-7.0339, 6.7123], Bias: -3.6412
Layer 3: 1 neurons
  Neuron 1: Weights: [12.7922, 12.9189], Bias: -6.3210

```

Figure 2: Final weights and bias for the XOR network after training.

The trained weights are very different from the hand-crafted ones in lab 4, but the function it compute is still XOR. This show that the backpropagation can find a working solution by itself, without we set the parameters by hand.

3.2 House price

For a more realistic test, I also train the network on a small house price dataset. The CSV have a few input features (for example: area, number of rooms, ...) and one output (the price). The price is normalized to $[0, 1]$ so the sigmoid output layer can fit it.

```

Training on lr.csv with backpropagation + gradient descent...
Normalization: minmax. input_scale=[(10.0, 80.0)], output_scale=[(55.0, 150.0)]
Training with loss = MSE, lr = 0.5, epochs = 10000
Epoch      1 / 10000 - loss: 0.450927
Epoch    1000 / 10000 - loss: 0.018418
Epoch    2000 / 10000 - loss: 0.017559
Epoch    3000 / 10000 - loss: 0.016735
Epoch    4000 / 10000 - loss: 0.015821
Epoch    5000 / 10000 - loss: 0.014627
Epoch    6000 / 10000 - loss: 0.013216
Epoch    7000 / 10000 - loss: 0.011620
Epoch    8000 / 10000 - loss: 0.009987
Epoch    9000 / 10000 - loss: 0.008443
Epoch   10000 / 10000 - loss: 0.007077

```

Figure 3: Training log for the house price dataset – loss decrease over epochs.

```

Evaluation on training set:
input=[10.0]  target=[55.0]  predicted=['62.6255']  -> [63]
input=[20.0]  target=[80.0]  predicted=['74.9246']  -> [75]
input=[40.0]  target=[100.0] predicted=['99.7594']  -> [100]
input=[60.0]  target=[120.0] predicted=['123.1851']  -> [123]
input=[80.0]  target=[150.0] predicted=['144.3797']  -> [144]

```

Figure 4: Predicted price vs target price on the training set.

The loss go down steadily, which mean the same backpropagation code work for a regression-style task too, not only XOR.

4 Conclusion

The lab show that with only the standard Python library, we can implement backpropagation and gradient descent by ourself. The same small framework can learn XOR and also a small house price dataset, just by changing the CSV file and the `structure.txt`. The key part is the chain rule: each neuron know how to turn the upstream gradient into gradients for its own weights and bias, and how to pass the gradient further back.